おかいもん

家族でリアルタイムに共有できる買い物リストPWA

プロジェクト概要

家族で使っていた買い物リストアプリが使いづらくなったため、
自分たちで好きにカスタマイズできるPWAアプリを個人開発。2025年〜継続中。

v1.4 現行バージョン	**14** 開発フェーズ	**27** 機能仕様書	**11** 設計書
~60 ファイル数 (src/)	**0円** 月額運用コスト		

画面紹介

モバイルファースト設計のPWA。ホーム画面に追加すればネイティブアプリと同じ体験。

ホーム画面（共有タブ）

グループで共有するリスト一覧。左右スワイプで個人タブに切替。

ホーム画面（個人タブ）

自分だけのリスト。共有タブと独立して管理。

リスト詳細

アイテム一覧。カテゴリ別の色分け、チェック/スワイプ削除。

アイテム詳細

数量・単位・価格・メモ・URL・カテゴリ・リマインダーを設定。

壁紙設定画面

プリセット or カスタム画像から選択。

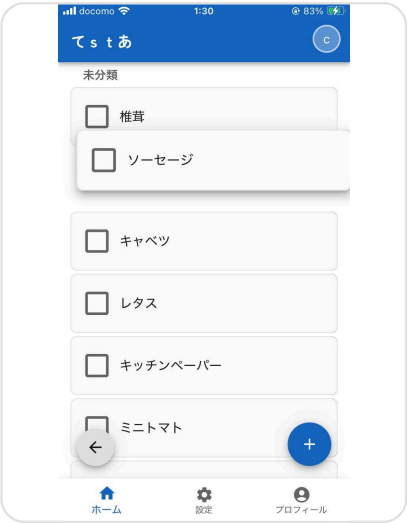
壁紙適用後

text-shadow + text-strokeでどんな背景でもテキスト視認性を確保。

操作系

スワイプ削除

左スワイプで削除ボタン表示。誤削除防止の2段階UX。



ドラッグ並び替え

長押しでドラッグ。浮動小数点ソートでDB書き込みコスト最小化。

通知設定

アイテム追加通知ON/OFF、通知不可時間帯の設定。

FABメニュー（ホーム）

リスト作成・並び替え・選択削除。＋ボタンに近い順にポジティブ操作。

FABメニュー（リスト内）

アイテム追加・並び替え・一括削除。破壊的操作は最上部に配置。

システム構成

```
┌──────────────────────────────────────────┐
│          クライアント（PWA / React）          │
│  Service Worker + IndexedDB（オフライン対応）  │
└──────────────────────────────────────────┘
       │ onSnapshot()                 │ Web Push
       │ （リアルタイム同期）            │
       ▼                              ▼

┌──────────────┐            ┌──────────────┐
│  Firestore   │            │ Cloud Functions │
│              │            │              │
│ users/       │  trigger   │ onItemCreate   │
│ groups/      │ ──────→    │  → まとめ通知    │
│ lists/       │            │ onReminderSet  │
│ items/       │            │  → Cloud Tasks  │
│              │            │ onReminderFire │
│ Security     │            │  → Push通知送信  │
│ Rules        │            └──────────────┘
│ (Custom      │                    │
│  Claims)     │                    ▼
└──────────────┘            ┌──────────────┐
                            │  Cloud Tasks   │
                            │ リマインダー予約  │
                            └──────────────┘

┌──────────────────────────────────────────┐
│            CI/CD（GitHub Actions）          │
│  develop → STG自動デプロイ                   │
│  main    → PROD自動デプロイ                  │
│  PR      → version-check（自動検証）         │
└──────────────────────────────────────────┘
```

データモデル

```
allowedUsers/{email}              招待制ホワイトリスト
users/{uid}                       ユーザー情報
  ├─ settings/preferences         設定（テーマ，通知，etc.）
  ├─ settings/wallpaperData       カスタム壁紙（Base64）
  ├─ devices/{deviceId}           Push通知トークン
  ├─ itemStats/{statId}           利用統計（サジェスト用）
  └─ lists/{listId}/items/{id}    個人リスト → アイテム

groups/{groupId}                  グループ情報
  └─ lists/{listId}/items/{id}    共有リスト → アイテム
```

技術スタック

FRONTEND

React 19 + TypeScript 5.9
MUI 7 (Material UI)
Vite 8 + vite-plugin-pwa
React Router 7
@dnd-kit (D&D)
Emotion (CSS-in-JS)

BACKEND (BAAS)

Firebase Auth (Google OAuth)
Cloud Firestore (NoSQL)
Cloud Functions (Node.js 22)
Cloud Tasks (リマインダー)
Firebase Hosting (CDN)

INFRASTRUCTURE

GitHub Actions (CI/CD)
STG / PROD 2環境自動デプロイ
Web Push API (通知)
Service Worker (オフライン)
husky (pre-push tsc -b)

主要機能

リアルタイム同期

Firestore onSnapshot でリスナー設定。家族の誰かがアイテムを追加すると全端末に即反映。Firestore Persistenceでオフライン時もローカル操作可能、復帰時に自動同期。

Firestore　onSnapshot

スワイプ2段階削除

1回目スワイプでチェック済み（灰色化）、2回目で削除確定。誤削除防止のUX設計。useSwipeToReveal カスタムhookで実装。

Custom Hook　Touch Events

ドラッグ&ドロップ並び替え

@dnd-kitで実装。浮動小数点ソート方式 — 並び替え時は移動アイテムの sortOrder を前後の中間値に設定 → 1 writeで完了（N件の連番振り直し不要）。

@dnd-kit　コスト最適化

プッシュ通知 + リマインダー

アイテム追加から5分間分をまとめて1通知に集約（Cloud Functions制御）。リマインダーはCloud Tasksで指定時刻にサーバー関数を起動 → プッシュ通知送信。

Cloud Functions　Cloud Tasks

認証 + PINロック

Firebase Auth (Google OAuth) + Custom ClaimsでJWTにグループ情報埋め込み。Web Credentials APIによる生体認証ロック対応。

Custom Claims　Web Credentials

壁紙・テーマカスタマイズ

プリセット壁紙 + カスタム画像アップロード（トリミング対応）。text-shadow + text-strokeの組み合わせで、どんな背景でもテキスト視認性を確保。

react-easy-crop　CSS

サジェスト

itemStats サブコレクションにアイテムの使用回数を記録。入力時に過去の使用頻度順でサジェスト表示。

統計ベース

招待制グループ

allowedUsers コレクションによるホワイトリスト制御。招待コード共有でグループ参加。Firestoreルールでグループメンバーの読み書きを制限。

Firestore Rules

技術的チャレンジ

通知のまとめ送信

課題: 買い物リストは短時間に複数アイテムを連続追加する → アイテムごとに通知すると大量通知になる

解決: Cloud Functions のトリガーで5分間の「まとめ待ち」を実装。最初のアイテム追加から5分後に、その間の追加分をまとめて1通知で送信。Cloud Tasksのスケジューリングを活用。

ドラッグ並び替えの書き込みコスト最小化

課題: Firestoreは書き込み回数で課金 → 並び替えでN件全部書き直すとコスト増

解決: 浮動小数点ソート方式を採用。並び替え時は移動アイテムの sortOrder を前後の中間値に設定するだけ → 1 write で完了。連番方式と比較してN-1回の書き込みを削減。

リマインダーの実行基盤（コスト0円）

課題: 「明日の朝8時に通知」を実現するには、サーバー側で指定時刻に処理を起動する必要がある

解決: Cloud Tasksでスケジュール登録 → 指定時刻にCloud Functionsを呼び出し → Web Push APIで通知送信。全てFirebase Sparkプラン（無料枠）内で実現。

オフライン対応とリアルタイム同期の両立

課題: PWAとしてオフラインでも動作しつつ、オンライン時はリアルタイム同期したい

解決: Firestore Persistenceを有効化。オフライン時はIndexedDBのローカルキャッシュで操作、ネットワーク復帰時にFirestoreが自動で差分同期。forceOwnership: true でタブ間のロック残り対策も実装。

壁紙設定時のテキスト視認性

課題: ユーザーが自由に壁紙を設定できる → 明るくても暗くても読めるようにする必要がある

解決: text-shadow（ぼかし影）+ text-stroke（縁取り）の組み合わせで、どんな背景色でも視認性を確保。白背景用のプリセットも追加。

運用コスト0円の維持

課題: 家族向けアプリなのでランニングコストをかけたくない

解決: Firebase Sparkプラン無料枠に収まる設計。まとめ通知でサーバー関数の呼び出し回数を削減、浮動小数点ソートでDB書き込みを削減、コスト監視アラートで無料枠超過を防止。

開発プロセス

仕様書駆動開発

全ての機能追加・変更をチケット番号付き仕様書（Markdown）で管理。累計27件の仕様書 + 11件の設計書。

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5
方針承認	**仕様書作成**	**仕様書確認**	**実装**	**デプロイ**
設計方針の合意	Markdown仕様書	レビュー・承認	コーディング	CI/CD自動デプロイ

CI/CD

自動デプロイ

develop push → STG環境に自動デプロイ
main push → PROD環境に自動デプロイ
Cloud Functions / Firestoreルール含む

品質チェック

pre-push hook: tsc -b (型チェック)
PR時: version-check (バージョン・リリースノート検証)
ESLint: コード品質

GIT運用

main ← **develop** ← **feat/** の3層構成
PRベースのマージ
featureブランチはマージ後に削除